

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv("learning_path_dataset.csv")

print("Dataset loaded successfully!")
```

Dataset loaded successfully!

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   student_id            1000 non-null   int64
1   study_hours           1000 non-null   float64
2   quiz_score            1000 non-null   float64
3   completed_topics      1000 non-null   int64
4   total_topics          1000 non-null   int64
5   practice_time         1000 non-null   float64
6   attendance            1000 non-null   float64
7   skill_level           1000 non-null   object
8   learning_goal         1000 non-null   object
9   course_difficulty     1000 non-null   object
10  recommended_topic     1000 non-null   object
dtypes: float64(4), int64(3), object(4)
memory usage: 86.1+ KB
```

```
print("Shape:", df.shape)
```

Shape: (1000, 11)

```
print("First 5 records:")
display(df.head())
```

First 5 records:

	student_id	study_hours	quiz_score	completed_topics	total_topics	practice_time	attendance	skill_level	learning_goal
0	1	7.08	78.15	7	15	1.82	98.95	Advanced	Data Science
1	2	1.40	70.99	13	15	4.01	65.41	Intermediate	Machine Learning
2	3	4.09	99.48	11	15	3.88	94.72	Beginner	Web Development
3	4	6.80	83.24	0	15	0.42	53.52	Beginner	Machine Learning
4	5	3.56	50.93	7	15	4.45	71.30	Beginner	Python

```
print("Last 5 records:")
display(df.tail())
```

Last 5 records:

	student_id	study_hours	quiz_score	completed_topics	total_topics	practice_time	attendance	skill_level	learning_goal
995	996	6.08	67.99	13	15	3.59	67.02	Beginner	Data Science
996	997	4.27	75.17	1	15	6.87	60.54	Intermediate	Machine Learning
997	998	5.16	47.76	8	15	2.19	77.81	Beginner	Data Science
998	999	8.22	64.64	2	15	0.51	53.80	Beginner	Data Science
999	1000	5.84	72.41	14	15	7.92	66.13	Intermediate	Machine Learning

```
print("\nData types:")
print(df.dtypes)
```

Data types:

student_id	int64
study_hours	float64
quiz_score	float64
completed_topics	int64
total_topics	int64
practice_time	float64
attendance	float64
skill_level	object
learning_goal	object
course_difficulty	object
recommended_topic	object

dtype: object

```
print("\nColumn names:")
print(df.columns.tolist())
```

Column names:

['student_id', 'study_hours', 'quiz_score', 'completed_topics', 'total_topics', 'practice_time', 'attendance', 'skill_level', 'learning_goal', 'course_difficulty', 'recommended_topic']

```
print("Statistical summary:")
display(df.describe())
```

Statistical summary:

	student_id	study_hours	quiz_score	completed_topics	total_topics	practice_time	attendance	
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.0	1000.000000	1000.000000	
mean	500.500000	5.417220	64.083470	7.027000	15.0	4.146090	75.263570	
std	288.819436	2.651007	20.133708	4.280724	0.0	2.350672	14.521838	
min	1.000000	1.010000	30.020000	0.000000	15.0	0.000000	50.030000	
25%	250.750000	3.140000	46.950000	3.000000	15.0	2.125000	62.722500	
50%	500.500000	5.385000	63.060000	7.000000	15.0	4.150000	75.330000	
75%	750.250000	7.742500	81.245000	11.000000	15.0	6.220000	88.117500	
max	1000.000000	9.980000	99.990000	14.000000	15.0	8.000000	99.920000	

```
print("\nUnique values:")
print(df.nunique())
```

Unique values:

student_id	1000
study_hours	600
quiz_score	935
completed_topics	15
total_topics	1

```

practice_time      574
attendance          912
skill_level         3
learning_goal       4
course_difficulty   3
recommended_topic    4
dtype: int64

```

```

print("Missing values:")
print(df.isnull().sum())

```

```

Missing values:
student_id      0
study_hours     0
quiz_score      0
completed_topics 0
total_topics    0
practice_time   0
attendance      0
skill_level     0
learning_goal   0
course_difficulty 0
recommended_topic 0
dtype: int64

```

```

print("\nMissing value percentage:")
print((df.isnull().sum() / len(df) * 100).round(2))

```

```

Missing value percentage:
student_id      0.0
study_hours     0.0
quiz_score      0.0
completed_topics 0.0
total_topics    0.0
practice_time   0.0
attendance      0.0
skill_level     0.0
learning_goal   0.0
course_difficulty 0.0
recommended_topic 0.0
dtype: float64

```

```

print("Duplicate records:", df.duplicated().sum())

df = df.drop_duplicates().reset_index(drop=True)

print("Shape after duplicate removal:", df.shape)

```

```

Duplicate records: 0
Shape after duplicate removal: (1000, 11)

```

```

numeric_cols = df.select_dtypes(
    include=np.number
).columns

categorical_cols = df.select_dtypes(
    exclude=np.number
).columns

for col in numeric_cols:
    df[col] = df[col].fillna(df[col].median())

for col in categorical_cols:
    df[col] = df[col].fillna(df[col].mode()[0])

print("Missing values after treatment:")
print(df.isnull().sum())

```

```

Missing values after treatment:
student_id      0
study_hours     0

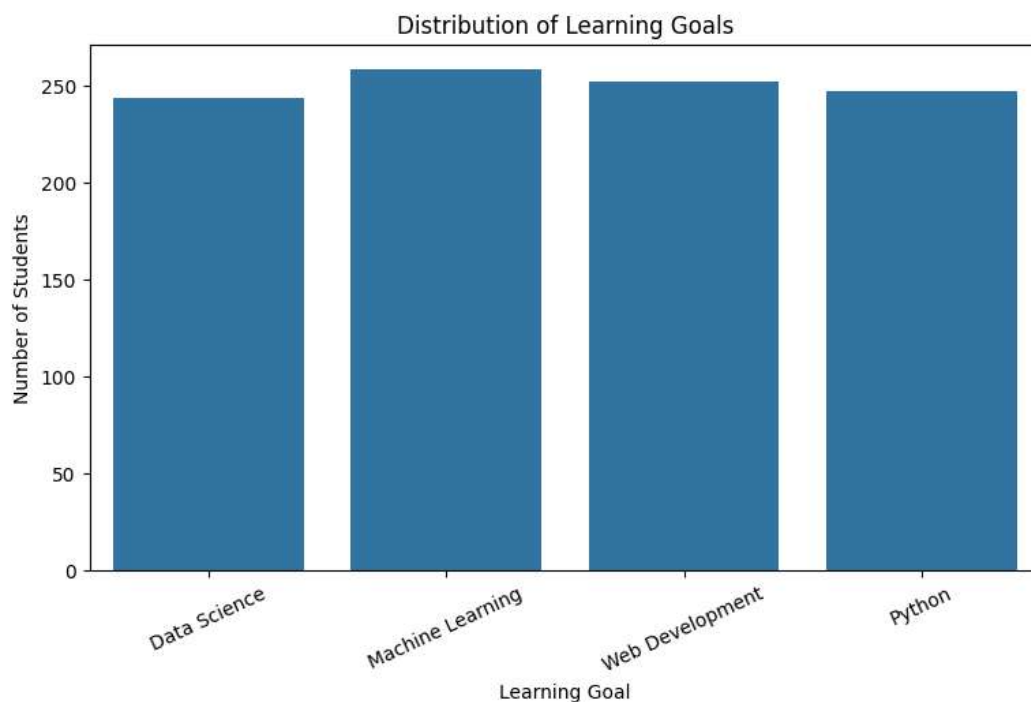
```

```
quiz_score      0
completed_topics 0
total_topics    0
practice_time    0
attendance       0
skill_level      0
learning_goal    0
course_difficulty 0
recommended_topic 0
dtype: int64
```

```
plt.figure(figsize=(9, 5))

sns.countplot(
    data=df,
    x="learning_goal"
)

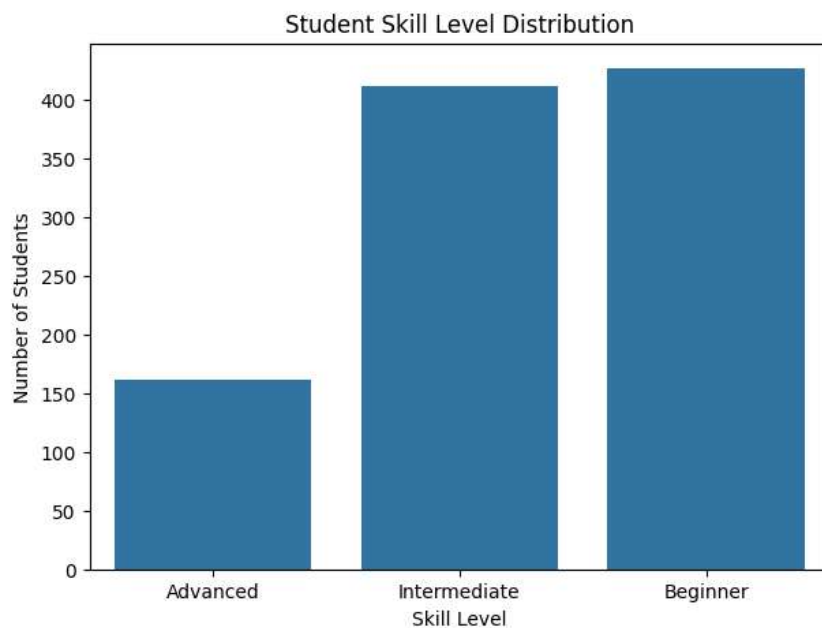
plt.title("Distribution of Learning Goals")
plt.xlabel("Learning Goal")
plt.ylabel("Number of Students")
plt.xticks(rotation=25)
plt.show()
```



```
plt.figure(figsize=(7, 5))

sns.countplot(
    data=df,
    x="skill_level"
)

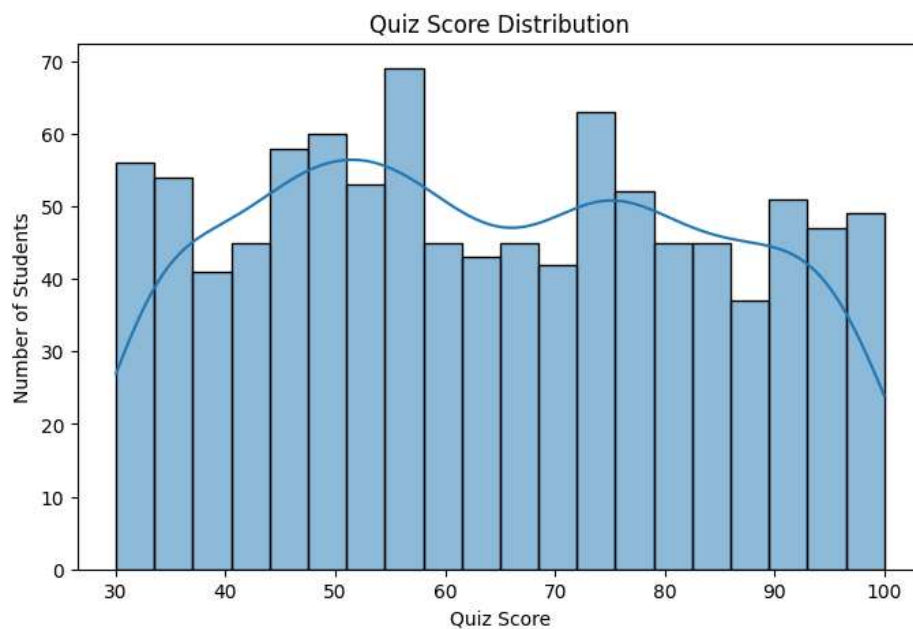
plt.title("Student Skill Level Distribution")
plt.xlabel("Skill Level")
plt.ylabel("Number of Students")
plt.show()
```



```
plt.figure(figsize=(8, 5))

sns.histplot(
    data=df,
    x="quiz_score",
    bins=20,
    kde=True
)

plt.title("Quiz Score Distribution")
plt.xlabel("Quiz Score")
plt.ylabel("Number of Students")
plt.show()
```

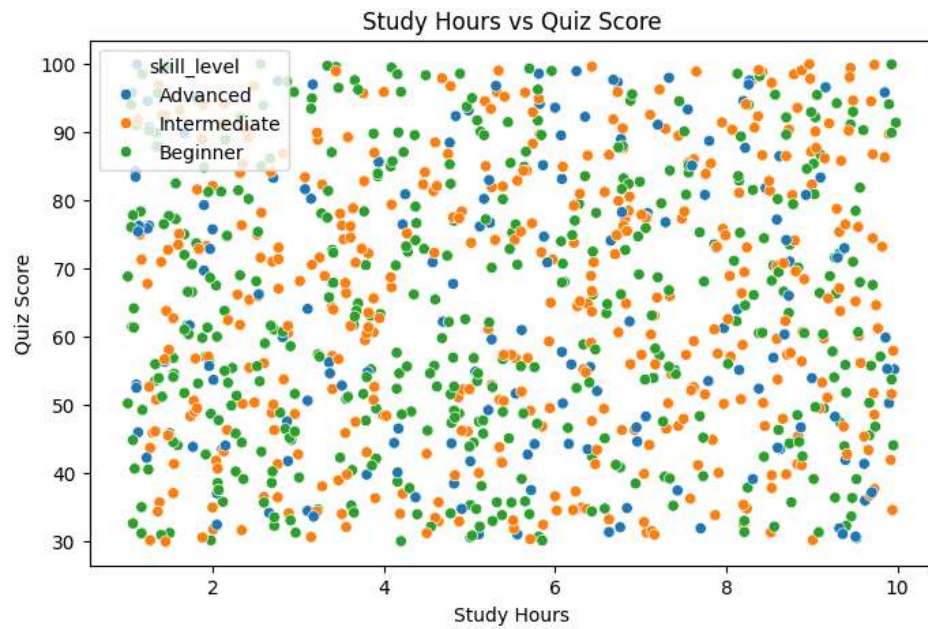


```
plt.figure(figsize=(8, 5))

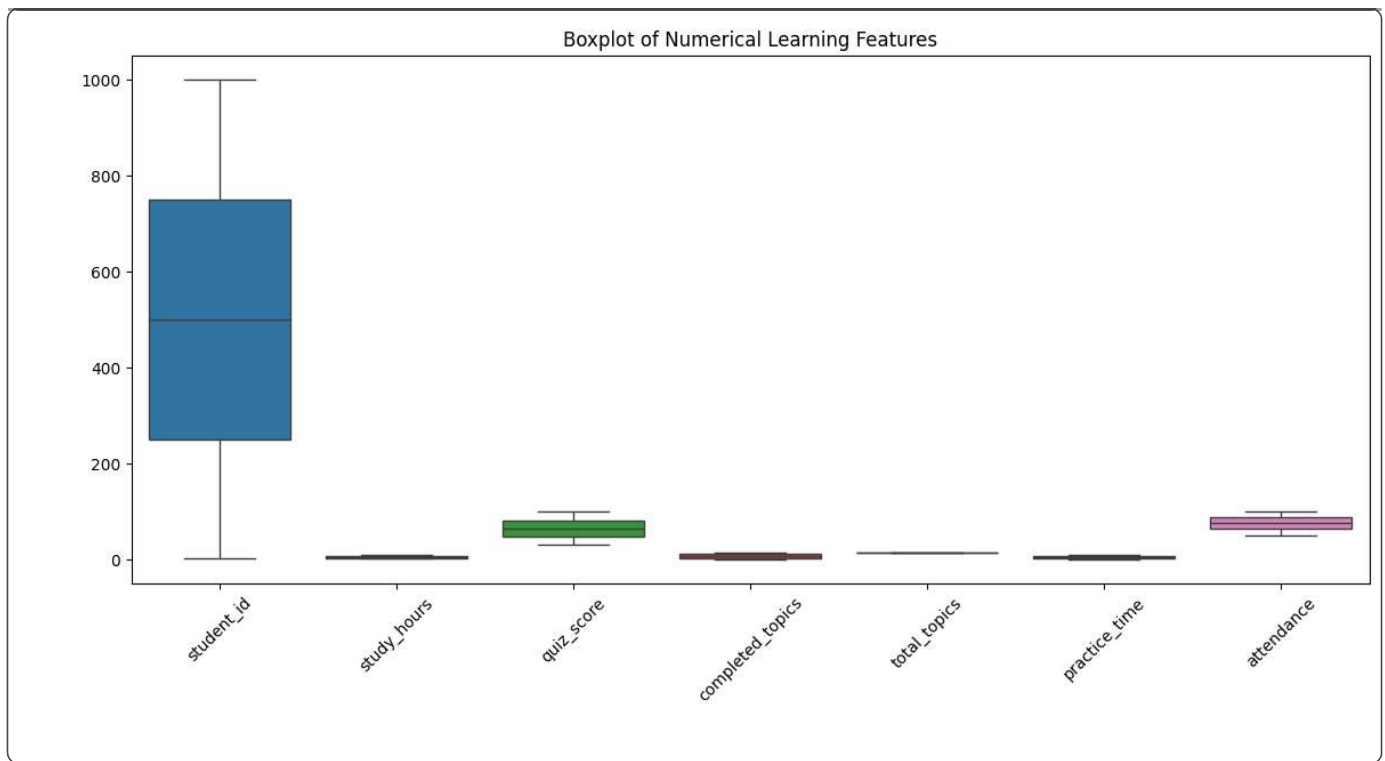
sns.scatterplot(
    data=df,
    x="study_hours",
    y="quiz_score",

```

```
    hue="skill_level"  
)  
  
plt.title("Study Hours vs Quiz Score")  
plt.xlabel("Study Hours")  
plt.ylabel("Quiz Score")  
plt.show()
```



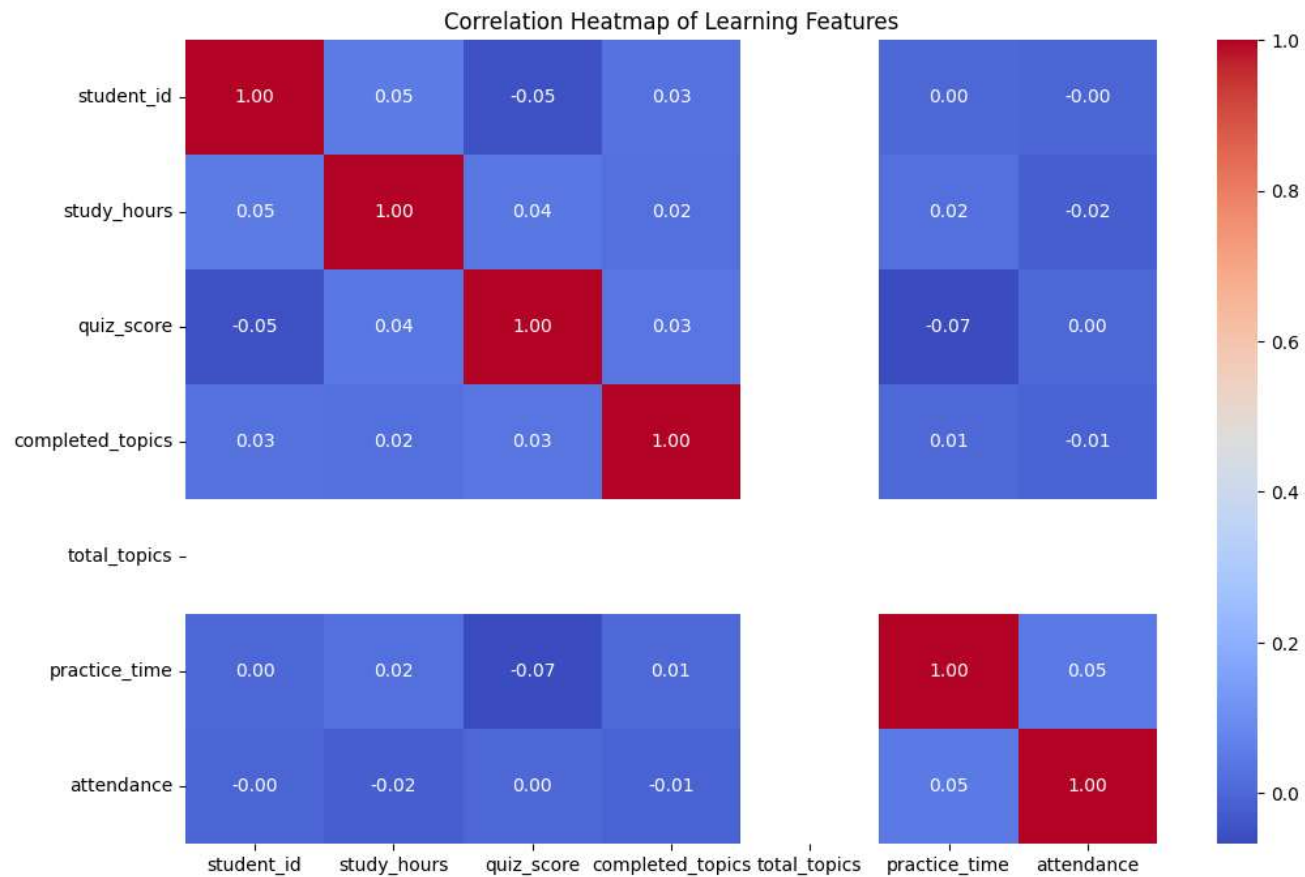
```
numeric_cols = df.select_dtypes(  
    include=np.number  
)<div data-bbox="125 603 460 603" data-label="Text">  
plt.figure(figsize=(14, 6))  
  
sns.boxplot(data=df[numeric_cols])  
  
plt.title("Boxplot of Numerical Learning Features")  
plt.xticks(rotation=45)  
plt.show()
```



```
plt.figure(figsize=(12, 8))

sns.heatmap(
    df[numeric_cols].corr(),
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap of Learning Features")
plt.show()
```



```
from scipy.stats import zscore

numeric_cols = df.select_dtypes(
    include=np.number
).columns.tolist()

z_scores = np.abs(
    zscore(df[numeric_cols])
)

outlier_count = (z_scores > 3).sum(axis=0)

outlier_df = pd.DataFrame({
    "Feature": numeric_cols,
    "Outlier Count": outlier_count
})

display(outlier_df)
```

/tmp/ipykernel_1451/554608358.py:8: RuntimeWarning: Precision loss occurred in moment calculation due to catastrophic cancell
zscore(df[numeric_cols])

	Feature	Outlier Count	
0	student_id	0	
1	study_hours	0	
2	quiz_score	0	
3	completed_topics	0	
4	total_topics	0	
5	practice_time	0	
6	attendance	0	

Next steps:

Generate code with outlier_df

New interactive sheet

```
df["progress_percentage"] = (
    df["completed_topics"] /
    df["total_topics"] * 100
)

df["learning_efficiency"] = (
    df["quiz_score"] /
    (df["study_hours"] + 1)
)

df["engagement_score"] = (
    df["study_hours"] +
    df["practice_time"] +
    df["attendance"] / 10
)

df["high_performer"] = (
    df["quiz_score"] >= 85
).astype(int)

print("Feature engineering completed!")

display(df.head())
```

Feature engineering completed!

	student_id	study_hours	quiz_score	completed_topics	total_topics	practice_time	attendance	skill_level	learning_goal
0	1	7.08	78.15	7	15	1.82	98.95	Advanced	Data Science
1	2	1.40	70.99	13	15	4.01	65.41	Intermediate	Machine Learning
2	3	4.09	99.48	11	15	3.88	94.72	Beginner	Web Development
3	4	6.80	83.24	0	15	0.42	53.52	Beginner	Machine Learning
4	5	3.56	50.93	7	15	4.45	71.30	Beginner	Python

```
numeric_cols = df.select_dtypes(
    include=np.number
).columns.tolist()

skewness = df[numeric_cols].skew()

print("Skewness of numerical features:")
print(skewness.sort_values(ascending=False))

plt.figure(figsize=(12, 5))

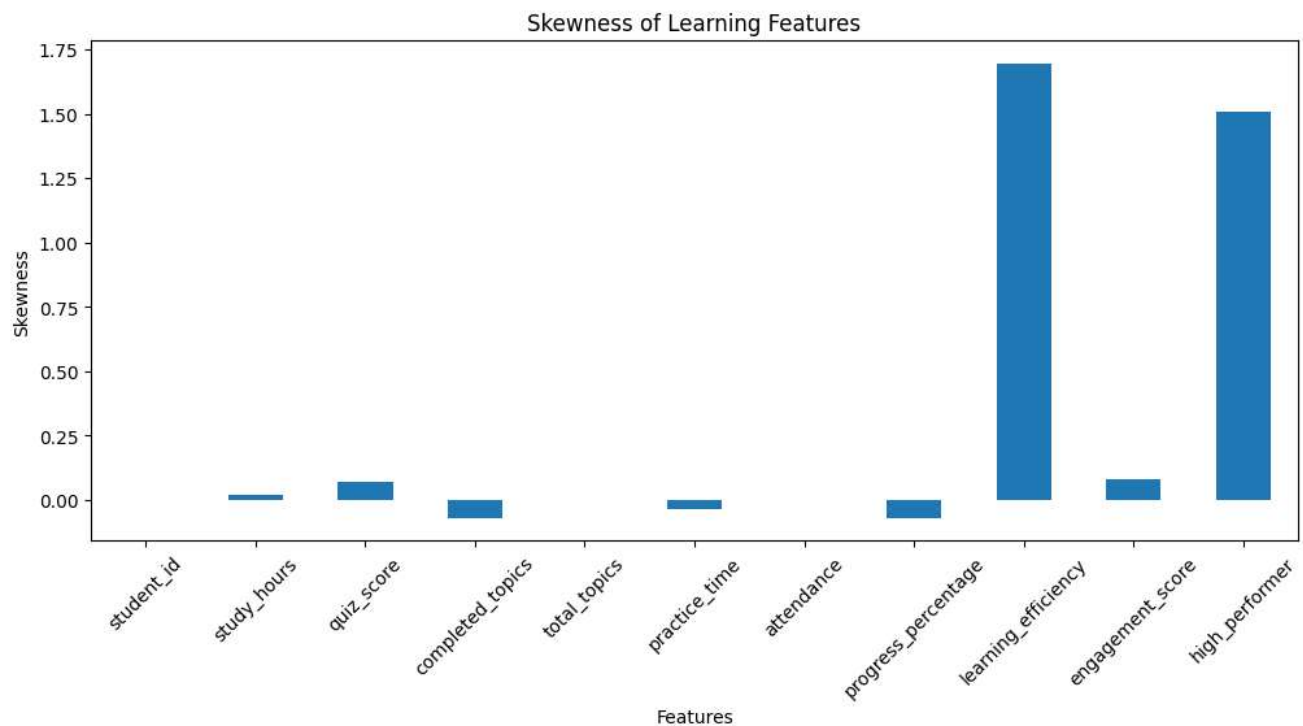
skewness.plot(kind="bar")

plt.title("Skewness of Learning Features")
plt.xlabel("Features")
plt.ylabel("Skewness")
plt.xticks(rotation=45)
plt.show()
```

```

Skewness of numerical features:
learning_efficiency    1.695191
high_performer         1.510101
engagement_score       0.076405
quiz_score             0.069395
study_hours            0.017405
student_id             0.000000
total_topics           0.000000
attendance             -0.001074
practice_time          -0.037051
progress_percentage    -0.072242
completed_topics       -0.072242
dtype: float64

```



```

from sklearn.preprocessing import PowerTransformer, StandardScaler

numeric_cols = df.select_dtypes(
    include=np.number
).columns.tolist()

# Exclude student ID
numeric_cols = [
    col for col in numeric_cols
    if col != "student_id"
]

power = PowerTransformer(
    method="yeo-johnson"
)

transformed_data = power.fit_transform(
    df[numeric_cols]
)

transformed_df = pd.DataFrame(
    transformed_data,
    columns=numeric_cols
)

print("After PowerTransformer:")
display(transformed_df.head())

scaler = StandardScaler()

scaled_data = scaler.fit_transform(

```

```
transformed_df
)

scaled_df = pd.DataFrame(
    scaled_data,
    columns=numeric_cols
)

print("After StandardScaler:")
display(scaled_df.head())
```

After PowerTransformer:

	study_hours	quiz_score	completed_topics	total_topics	practice_time	attendance	progress_percentage	learning_efficiency
0	0.656696	0.723226	0.063583	0.0	-0.971207	1.593400	0.100429	-0.09332
1	-1.613641	0.390698	1.322664	0.0	0.000835	-0.663678	1.278420	1.71541
2	-0.441982	1.654016	0.919902	0.0	-0.053630	1.320754	0.907412	1.09615
3	0.559601	0.952938	-1.796437	0.0	-1.688236	-1.529471	-1.926640	0.08297
4	-0.654587	-0.613295	0.063583	0.0	0.182825	-0.249558	0.100429	0.16352

After StandardScaler:

	study_hours	quiz_score	completed_topics	total_topics	practice_time	attendance	progress_percentage	learning_efficiency
0	0.656696	0.723226	0.063583	0.0	-0.971207	1.593400	0.100429	-0.09332
1	-1.613641	0.390698	1.322664	0.0	0.000835	-0.663678	1.278420	1.71541
2	-0.441982	1.654016	0.919902	0.0	-0.053630	1.320754	0.907412	1.09615
3	0.559601	0.952938	-1.796437	0.0	-1.688236	-1.529471	-1.926640	0.08297
4	-0.654587	-0.613295	0.063583	0.0	0.182825	-0.249558	0.100429	0.16352

```
from sklearn.preprocessing import LabelEncoder

df_encoded = df.copy()

categorical_cols = df_encoded.select_dtypes(
    exclude=np.number
).columns.tolist()

encoders = {}

for col in categorical_cols:
    le = LabelEncoder()
    df_encoded[col] = le.fit_transform(
        df_encoded[col].astype(str)
    )
    encoders[col] = le

print("Label encoding completed!")

display(df_encoded.head())
```

Label encoding completed!

	student_id	study_hours	quiz_score	completed_topics	total_topics	practice_time	attendance	skill_level	learning_goal
0	1	7.08	78.15	7	15	1.82	98.95	0	0
1	2	1.40	70.99	13	15	4.01	65.41	2	1
2	3	4.09	99.48	11	15	3.88	94.72	1	3
3	4	6.80	83.24	0	15	0.42	53.52	1	1
4	5	3.56	50.93	7	15	4.45	71.30	1	2

```
from sklearn.model_selection import train_test_split

X = df.drop(
    columns=["recommended_topic", "student_id"]
```

```

)

y = df["recommended_topic"]

label_encoder = LabelEncoder()

y_encoded = label_encoder.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.30,
    random_state=51,
    stratify=y_encoded
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)

```

Training data: (700, 13)
Testing data: (300, 13)

```

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import (
    OneHotEncoder,
    StandardScaler
)

numeric_features = X_train.select_dtypes(
    include=np.number
).columns.tolist()

categorical_features = X_train.select_dtypes(
    exclude=np.number
).columns.tolist()

numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(
        handle_unknown="ignore",
        sparse_output=False
    ))
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)

print("Preprocessing pipeline created!")

```

Preprocessing pipeline created!

```

from imblearn.over_sampling import SMOTE

# First transform training data
X_train_processed = preprocessor.fit_transform(X_train)

print("Before SMOTE:", X_train_processed.shape)

smote = SMOTE(random_state=51)

X_train_balanced, y_train_balanced = smote.fit_resample(
    X_train_processed,

```

```

    y_train
)

print("After SMOTE:", X_train_balanced.shape)

print("\nClass distribution after SMOTE:")
print(pd.Series(y_train_balanced).value_counts())

```

Before SMOTE: (700, 20)
After SMOTE: (828, 20)

Class distribution after SMOTE:

```

1    207
2    207
3    207
0    207
Name: count, dtype: int64

```

```

from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC

from sklearn.ensemble import (
    RandomForestClassifier,
    AdaBoostClassifier,
    GradientBoostingClassifier
)

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Gaussian Naive Bayes": GaussianNB(),
    "Decision Tree": DecisionTreeClassifier(random_state=51),
    "KNN": KNeighborsClassifier(),
    "SVM": SVC(probability=True, random_state=51),
    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        random_state=51,
        n_jobs=-1
    ),
    "AdaBoost": AdaBoostClassifier(random_state=51),
    "Gradient Boosting": GradientBoostingClassifier(random_state=51)
}

results = []
trained_models = {}

for name, model in models.items():

    model.fit(X_train_balanced, y_train_balanced)

    pred = model.predict(
        preprocessor.transform(X_test)
    )

    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, pred),
        "Precision": precision_score(
            y_test, pred, average="weighted", zero_division=0
        ),
        "Recall": recall_score(
            y_test, pred, average="weighted", zero_division=0
        ),
        "F1 Score": f1_score(
            y_test, pred, average="weighted", zero_division=0
        )
    })

```

```

    })

    trained_models[name] = model

results_df = pd.DataFrame(results)

display(results_df.sort_values(
    by="F1 Score",
    ascending=False
))

```

	Model	Accuracy	Precision	Recall	F1 Score
2	Decision Tree	1.000000	1.000000	1.000000	1.000000
5	Random Forest	1.000000	1.000000	1.000000	1.000000
7	Gradient Boosting	1.000000	1.000000	1.000000	1.000000
1	Gaussian Naive Bayes	0.980000	0.980134	0.980000	0.979993
0	Logistic Regression	0.970000	0.970080	0.970000	0.969847
4	SVM	0.953333	0.953479	0.953333	0.953256
3	KNN	0.756667	0.757949	0.756667	0.753094
6	AdaBoost	0.486667	0.302722	0.486667	0.351247

```

from sklearn.metrics import (
    confusion_matrix,
    classification_report
)

best_name = results_df.sort_values(
    by="F1 Score",
    ascending=False
).iloc[0]["Model"]

best_model = trained_models[best_name]

pred = best_model.predict(
    preprocessor.transform(X_test)
)

print("Best Model:", best_name)

print("\nClassification Report:")
print(classification_report(
    y_test,
    pred,
    target_names=label_encoder.classes_,
    zero_division=0
))

cm = confusion_matrix(y_test, pred)

plt.figure(figsize=(7, 6))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=label_encoder.classes_,
    yticklabels=label_encoder.classes_
)

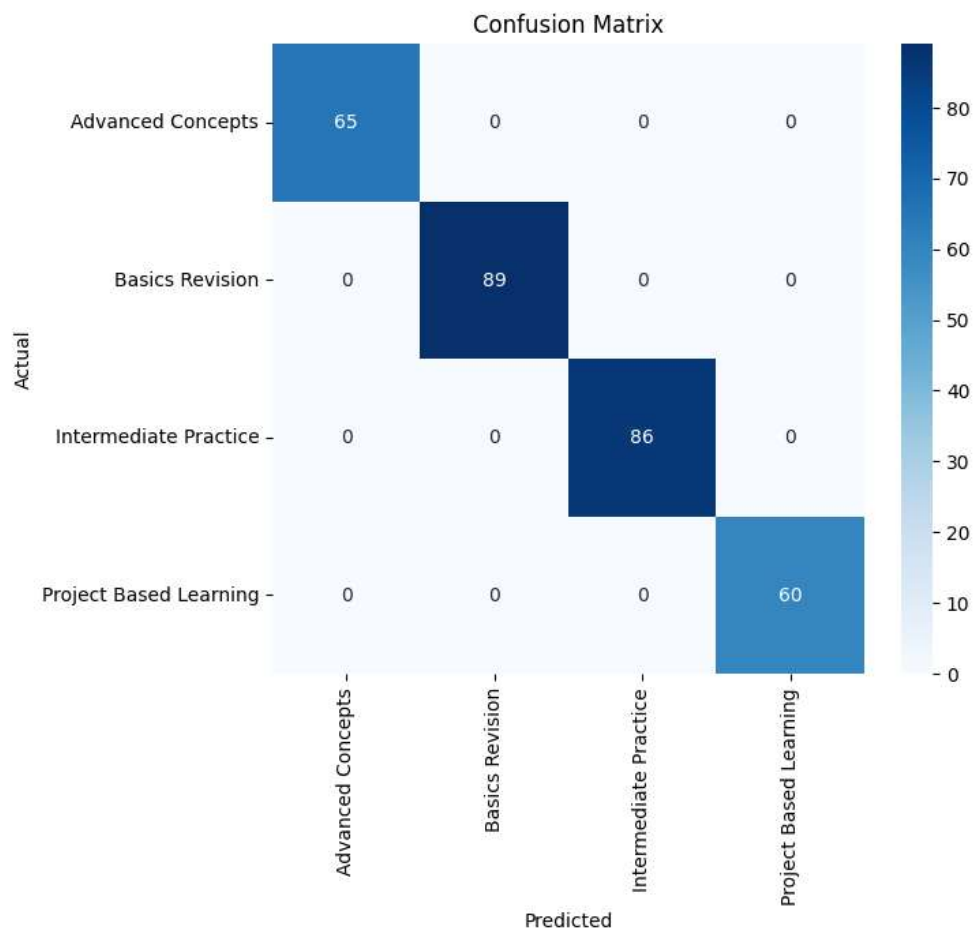
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

```

Best Model: Decision Tree

Classification Report:

	precision	recall	f1-score	support
Advanced Concepts	1.00	1.00	1.00	65
Basics Revision	1.00	1.00	1.00	89
Intermediate Practice	1.00	1.00	1.00	86
Project Based Learning	1.00	1.00	1.00	60
accuracy			1.00	300
macro avg	1.00	1.00	1.00	300
weighted avg	1.00	1.00	1.00	300



```

from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_auc_score, roc_curve

y_test_binary = label_binarize(
    y_test,
    classes=np.arange(len(label_encoder.classes_))
)

probabilities = best_model.predict_proba(
    preprocessor.transform(X_test)
)

roc_auc = roc_auc_score(
    y_test_binary,
    probabilities,
    multi_class="ovr",
    average="weighted"
)

print("Weighted ROC-AUC:", roc_auc)

plt.figure(figsize=(8, 6))

```

```

for i, class_name in enumerate(label_encoder.classes_):
    fpr, tpr, _ = roc_curve(
        y_test_binary[:, i],
        probabilities[:, i]
    )

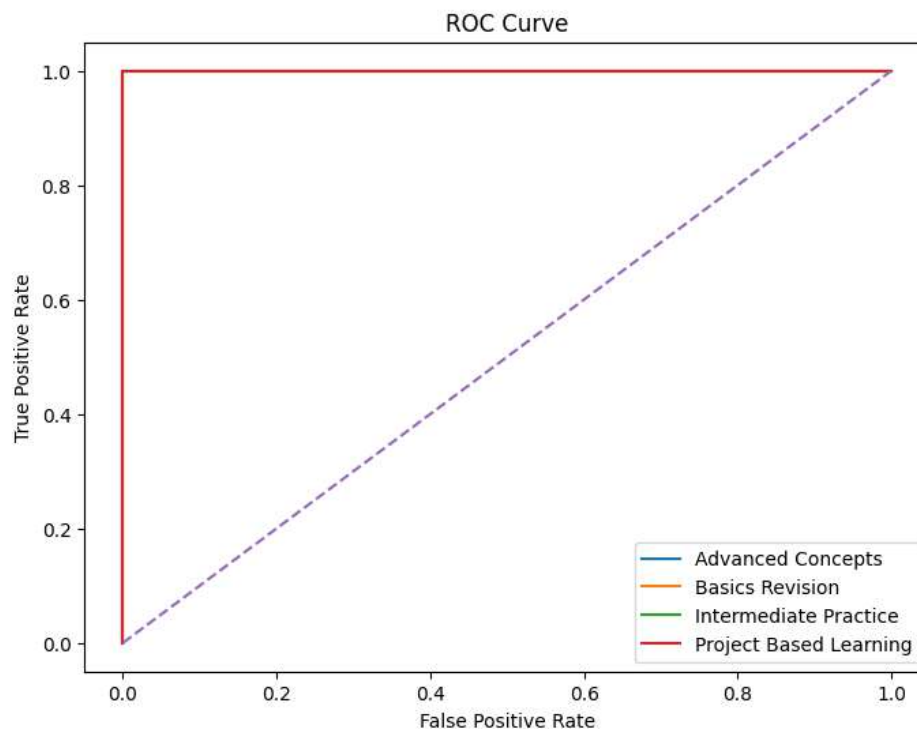
    plt.plot(
        fpr,
        tpr,
        label=class_name
    )

plt.plot([0, 1], [0, 1], linestyle="--")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()

```

Weighted ROC-AUC: 1.0



```

from sklearn.model_selection import StratifiedKFold, cross_val_score

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=51
)

cv_results = []

for name, model in models.items():

    scores = cross_val_score(
        model,
        X_train_balanced,
        y_train_balanced,
        cv=cv,
        scoring="f1_weighted"
    )

    cv_results.append({
        "Model": name,
        "Mean CV F1": scores.mean(),
    })

```

```

        "Std CV F1": scores.std()
    })

cv_df = pd.DataFrame(cv_results)

display(cv_df.sort_values(
    by="Mean CV F1",
    ascending=False
))

```

	Model	Mean CV F1	Std CV F1	
2	Decision Tree	0.997590	0.002952	
7	Gradient Boosting	0.997590	0.002952	
5	Random Forest	0.997583	0.002960	
0	Logistic Regression	0.969725	0.016774	
1	Gaussian Naive Bayes	0.964750	0.017974	
4	SVM	0.930280	0.018251	
3	KNN	0.779965	0.009180	
6	AdaBoost	0.370439	0.015211	

```

from sklearn.model_selection import GridSearchCV

rf = RandomForestClassifier(
    random_state=51,
    n_jobs=-1
)

param_grid = {
    "n_estimators": [100, 200],
    "max_depth": [None, 10, 20],
    "min_samples_split": [2, 5],
    "min_samples_leaf": [1, 2]
}

grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    cv=3,
    scoring="f1_weighted",
    n_jobs=-1,
    verbose=1
)

grid_search.fit(
    X_train_balanced,
    y_train_balanced
)

print("Best Parameters:")
print(grid_search.best_params_)

print("\nBest CV Score:")
print(grid_search.best_score_)

```

Fitting 3 folds for each of 24 candidates, totalling 72 fits

Best Parameters:
{'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 5, 'n_estimators': 200}

Best CV Score:
0.9987922071102243

```

tuned_model = grid_search.best_estimator_

tuned_pred = tuned_model.predict(
    preprocessor.transform(X_test)
)

```

```
print("Tuned Random Forest Evaluation")

print("Accuracy:", accuracy_score(y_test, tuned_pred))

print("Precision:", precision_score(
    y_test, tuned_pred, average="weighted"
))

print("Recall:", recall_score(
    y_test, tuned_pred, average="weighted"
))

print("F1 Score:", f1_score(
    y_test, tuned_pred, average="weighted"
))
```

```
Tuned Random Forest Evaluation
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0
```

```
feature_names = preprocessor.get_feature_names_out()

importance_df = pd.DataFrame({
    "Feature": feature_names,
    "Importance": tuned_model.feature_importances_
})

importance_df = importance_df.sort_values(
    by="Importance",
    ascending=False
)

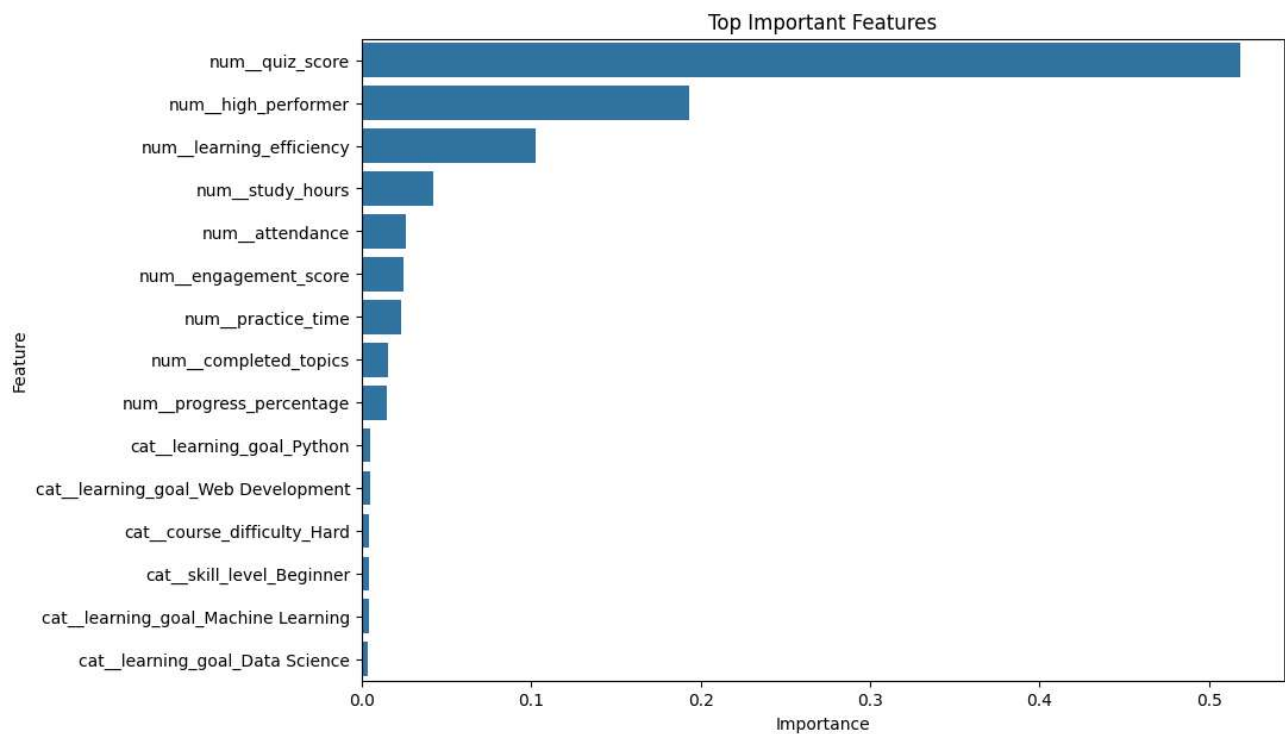
display(importance_df.head(15))

plt.figure(figsize=(10, 7))

sns.barplot(
    data=importance_df.head(15),
    x="Importance",
    y="Feature"
)

plt.title("Top Important Features")
plt.show()
```

	Feature	Importance
1	num__quiz_score	0.518361
9	num__high_performer	0.192939
7	num__learning_efficiency	0.102703
0	num__study_hours	0.042327
5	num__attendance	0.026145
8	num__engagement_score	0.024664
4	num__practice_time	0.023329
2	num__completed_topics	0.015721
6	num__progress_percentage	0.015108
15	cat__learning_goal_Python	0.004996
16	cat__learning_goal_Web Development	0.004756
18	cat__course_difficulty_Hard	0.004332
11	cat__skill_level_Beginner	0.004018
14	cat__learning_goal_Machine Learning	0.003981
13	cat__learning_goal_Data Science	0.003878



```

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC

# Dataset split - These are the original, untransformed splits.
# The error occurred because these were used directly without preprocessing.

```

```

# We will use the preprocessed and balanced data for training and evaluation.
# x_train, x_test, y_train, y_test = train_test_split(
#     X, y,
#     test_size=0.30,
#     random_state=51,
#     stratify=y
# )

# Re-split to ensure y_train and y_test are the numerical encoded versions
# This ensures consistency with y_encoded from cell 7qb6-fQK2Wge
_, _, y_train_numerical, y_test_numerical = train_test_split(
    X,
    y_encoded,
    test_size=0.30,
    random_state=51,
    stratify=y_encoded
)

# Models
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(random_state=51),
    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        random_state=51
    ),
    "Gradient Boosting": GradientBoostingClassifier(
        random_state=51
    ),
    "KNN": KNeighborsClassifier(),
    "Naive Bayes": GaussianNB(),
    "SVM": SVC()
}

results = []

for name, model in models.items():
    # Fit models on the balanced and preprocessed training data
    model.fit(X_train_balanced, y_train_balanced)

    # Predict on the preprocessed test data
    y_pred = model.predict(
        preprocessor.transform(X_test)
    )

    accuracy = accuracy_score(y_test_numerical, y_pred)
    precision = precision_score(
        y_test_numerical, y_pred, average="weighted", zero_division=0
    )
    recall = recall_score(
        y_test_numerical, y_pred, average="weighted", zero_division=0
    )
    f1 = f1_score(
        y_test_numerical, y_pred, average="weighted", zero_division=0
    )

    # Perform cross-validation on the balanced and preprocessed training data
    cv_score = cross_val_score(
        model, X_train_balanced, y_train_balanced, cv=5, scoring="accuracy"
    ).mean()

    results.append({
        "Model": name,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-Score": f1,
        "Cross Validation": cv_score
    })

# Comparison table
results_df = pd.DataFrame(results)

print("Model Performance Comparison:")
print(results_df.round(4))

```

```
# Best model based on F1-score
best_model_name = results_df.loc[
    results_df["F1-Score"].idxmax(), "Model"
]

best_model = models[best_model_name]

print("\nBest Model:", best_model_name)
print("Best F1-Score:",
      results_df["F1-Score"].max())
```

Model Performance Comparison:

	Model	Accuracy	Precision	Recall	F1-Score \
0	Logistic Regression	0.9700	0.9701	0.9700	0.9698
1	Decision Tree	1.0000	1.0000	1.0000	1.0000
2	Random Forest	1.0000	1.0000	1.0000	1.0000
3	Gradient Boosting	1.0000	1.0000	1.0000	1.0000
4	KNN	0.7567	0.7579	0.7567	0.7531
5	Naive Bayes	0.9800	0.9801	0.9800	0.9800
6	SVM	0.9533	0.9535	0.9533	0.9533

Cross Validation

0	0.9625
1	0.9976
2	0.9964
3	0.9964
4	0.7657
5	0.9625
6	0.9215

Best Model: Decision Tree

Best F1-Score: 1.0

```
import random

states = [
    "Beginner_Low",
    "Beginner_Medium",
    "Intermediate_Low",
    "Intermediate_Medium",
    "Advanced_High"
]

actions = [
    "Python Basics",
    "Python Practice",
    "Data Analysis",
    "Machine Learning",
    "Project Based Learning"
]

q_table = pd.DataFrame(
    0.0,
    index=states,
    columns=actions
)

alpha = 0.1
gamma = 0.9
epsilon = 0.2
episodes = 1000

def get_state(skill, score):

    if skill == "Beginner" and score < 60:
        return "Beginner_Low"

    elif skill == "Beginner":
        return "Beginner_Medium"

    elif skill == "Intermediate" and score < 70:
        return "Intermediate_Low"

    elif skill == "Intermediate":
        return "Intermediate_Medium"
```

```
    else:
        return "Advanced_High"

def get_reward(action, score, progress):

    reward = 0

    if action == "Python Basics" and score < 60:
        reward += 10

    if action == "Python Practice" and 60 <= score < 75:
        reward += 10

    if action == "Data Analysis" and 70 <= score < 85:
        reward += 10

    if action == "Machine Learning" and score >= 75:
```